Data Engineering — Scenario Questions

21 behavioural + real-time questions · my notes — Sagar · Jan 2026

How to answer (every time)

1. **Situation** — one line: system, scale, what broke
2. **Action** — exact technical steps, real tool names
3. **Result** — a number: runtime, cost, SLA, hours saved
4. **Prevention** — the check / alert / runbook I added

* swap placeholder numbers with my real ones — follow-ups will test them!

Production incidents & failures

Q1. Tell me about a major problem you faced and how you overcame it.

Our daily orders pipeline on Databricks started missing its 6 AM SLA, running 4+ hours instead of 90 minutes. In the Spark UI one stage had a single task running 40 minutes while the rest took seconds: data skew on a **seller_id** join. I enabled AQE skew join handling, salted the hot keys, broadcast the small dimension table, and compacted small files with **OPTIMIZE** + Z-order on the join key. Runtime dropped to ~50 minutes and we haven't missed the SLA since. I added a runtime alert at 120% of baseline. Before changing anything I confirmed it was skew, not volume: total input was flat but max task time was 60x the median. Salting splits each hot key into N sub-keys on the big side and explodes the small side N times, so the join result stays correct.

▶ **Spark UI → skew → AQE + salting + broadcast → alert**

> *If they push back:*
>
> ↳ **"Why not just add more executors?"**
> Skew means one task holds most of the data. More executors don't split that task, so the stage waits on it anyway. Fixing the distribution was the only real fix.
>
> ↳ **"When would AQE alone not be enough?"**
> AQE splits skewed partitions only in sort-merge joins and only above its thresholds. Extreme skew, or skew inside an aggregation, still needs salting or a two-stage aggregation.
>
> ↳ **"Any downside to salting?"**
> The small side is replicated N times, so I salt only the hot keys and pick N from the skew ratio, not a blanket number.

Q2. A pipeline failed at 3 AM and you are on call. Walk me through it.

Acknowledge the PagerDuty alert so it's owned. Check impact: which downstream tables and reports, and by when. Read driver/executor logs and classify: source data, infra (cluster, OOM, quota) or code. Transient failure → rerun the task from Airflow, safe

because writes are idempotent (MERGE / partition overwrite). Data issue → quarantine the bad batch, load the rest. Status update to stakeholders before business hours with an ETA. Next day: short RCA + the missing check or retry rule. I always decide rerun vs. fix-forward based on whether the failure is deterministic: if the same input will fail again, a rerun just burns cluster time.

▶ **Own → impact → classify → idempotent rerun → update → RCA**

> *If they push back:*
>
> ↳ **"How do you know a rerun won't create duplicates?"**
> Every write is idempotent: partition overwrite with replaceWhere or a MERGE on the business key. Running the same batch twice gives the same result.
>
> ↳ **"What if you can't fix it before the business day starts?"**
> I give consumers a clear status and ETA, and if possible publish yesterday's data with a staleness flag rather than partial or wrong data.
>
> ↳ **"What goes into your RCA?"**
> Timeline, root cause, impact, what fixed it, and one or two prevention actions with owners. It's blameless and about the system.

Q3. Bad data reached a business dashboard. What did you do?

An upstream team changed a currency column from rupees to paise without notice, so revenue showed 100x for one morning. I paused the downstream refresh and told the dashboard owners immediately. Backfilled the affected partitions, using Delta time travel to compare before/after. Added DQ checks that fail the job on range and volume anomalies (e.g. daily revenue outside ±3σ of last 30 days) and agreed a schema + semantics contract with the source team. The key decision was to pause the refresh first. Wrong numbers that stay up for hours do more damage than numbers that arrive late, because people make decisions on them.

▶ **Stop the spread first, then fix, then add DQ checks**

> *If they push back:*
>
> ↳ **"Why didn't your checks catch it?"**
> We had schema checks, and the type didn't change, only the unit. That's exactly why I added distribution and range checks, not just schema checks.
>
> ↳ **"Won't strict DQ checks cause false alarms?"**
> I tuned thresholds on 30 days of history and split checks into blocking (hard rules) and warning (soft anomalies), so only real breaks stop the pipeline.

Q4. Tell me about a mistake you made.

I ran a backfill with **mode('overwrite')** on a partitioned Delta table without **replaceWhere**; it replaced the full table with one week of data. I caught it from the row-count drop within ~20 minutes and told my lead right away. Restored with **RESTORE TABLE ... VERSION AS OF**, reran correctly, no data loss. Added a pre-write row-count guard to our write utility and made overwrite logic a mandatory code-review item. I'm honest about this

one because the recovery and the guardrail matter more than the mistake. Delta's transaction log made it recoverable; on plain Parquet or Hive it would have been a real data loss.

▶ **Report in 20 min > someone finds it in a day**

If they push back:

↳ **"Why didn't you test it first?"**
I tested the logic on a dev table, but the dev table wasn't partitioned the same way. Now dev mirrors prod partitioning, and backfills run on a cloned table first.

↳ **"What if time travel had already been vacuumed?"**
That's why VACUUM retention is set to 7+ days on critical tables, and we keep the raw bronze layer so any silver table can be rebuilt.

↳ **"What does the row-count guard do?"**
Before an overwrite, it compares rows being replaced vs. rows being written. A drop beyond a threshold aborts the write unless a flag explicitly allows it.

Q5. How did you handle duplicates in a streaming pipeline?

A deployment changed the query and its checkpoint location, so the Kafka consumer reprocessed offsets and loaded duplicates. I found the affected window from event timestamps and deduplicated on business key + event time with a window function, keeping the latest record. Permanent fix: idempotent sink using **foreachBatch** + Delta MERGE on the business key, plus **dropDuplicatesWithinWatermark**. Checkpoint changes now follow a runbook. Exactly-once in Spark streaming needs three things together: a replayable source like Kafka, checkpointed offsets, and an idempotent sink. If any one is missing, you get duplicates on a restart.

▶ **Checkpoint change = #1 cause of streaming dupes**

If they push back:

↳ **"Why did a new checkpoint cause reprocessing?"**
Offsets live in the checkpoint. A new location has no offsets, so the query started from startingOffsets, which replayed old messages.

↳ **"Why not just use dropDuplicates?"**
Plain dropDuplicates keeps state forever and grows without limit. With a watermark the state is bounded, and MERGE handles anything outside the window.

Performance, scale & cost

Q6. Your job ran fine for months and suddenly became slow. How do you debug it?

Compare a good run vs a bad run, don't guess. Check input volume (row and file counts per partition), then in Spark UI compare stage durations, shuffle, spill and task time distribution. In one case it was the small-file problem: upstream started writing thousands of 1 MB files, so listing and scheduling dominated. Compacted with **OPTIMIZE**, enabled optimized writes + auto-compaction. Back to baseline; now I track file count and avg file

size as metrics. I use the last successful run's metrics as a baseline, so the question becomes 'what changed' rather than 'why is it slow', which is much faster to answer.

▶ **Good run vs bad run → volume, skew, small files**

If they push back:

↳ **"How did you know it was small files?"**
Input file count went from hundreds to tens of thousands with average size near 1 MB, and the Spark UI showed long gaps before tasks started, which is listing and scheduling overhead.

↳ **"Downside of auto-compaction?"**
Writes take slightly longer. For a table read many times a day, that trade-off is worth it.

Q7. How did you reduce cloud cost on a pipeline?

A BigQuery reporting workload was scanning full tables. I partitioned by event date, clustered on common filter columns, set **require_partition_filter**, and replaced repeated heavy queries with incremental materialized tables. Bytes scanned dropped ~70%. On Databricks: job clusters instead of all-purpose, autoscaling, spot for non-critical jobs, auto-termination. I measured first: INFORMATION_SCHEMA jobs data showed which queries and users scanned the most bytes, so I fixed the top few instead of guessing.

▶ **Partition + cluster + force partition filter**

If they push back:

↳ **"How did you verify the 70% saving?"**
I compared total bytes billed for the same dashboards over two weeks before and after, using the jobs metadata.

↳ **"Doesn't require_partition_filter break existing queries?"**
It can, so I found affected queries in the job history first, fixed them with the owners, then turned it on.

Q8. An executor keeps failing with OutOfMemory. What do you do?

More memory is the last step. Driver OOM → usually **collect()**, a big broadcast, or listing too many small files. Executor OOM → usually skew or oversized partitions. Check spill and partition sizes in Spark UI, then fix the cause: partitions ~128–200 MB, AQE or salting for skew, replace **collect()** with writes/aggregations. Only then tune **spark.executor.memory** / memoryOverhead. Most OOMs I've seen were design problems, not memory problems, so throwing memory at them only delays the next failure.

▶ **Driver vs executor OOM — fix cause before memory**

If they push back:

↳ **"What is memoryOverhead for?"**
Off-heap memory for things like Python workers, network buffers and native libraries. PySpark jobs with UDFs often need more overhead, not more heap.

↳ **"How do you pick the number of partitions?"**
Target ~128–200 MB per partition: input size divided by that, and with AQE I let coalescing tune shuffle partitions at runtime.

Q9. How would you migrate on-prem Hadoop/Hive to the cloud without breaking downstream?

Phased, not big bang. Inventory tables, jobs, dependencies and consumers. Migrate one domain at a time and run old + new in parallel for a few cycles. Reconcile automatically: row counts, key-level checksums, aggregates on critical metrics. Switch consumers only after reconciliation passes for an agreed period, with a rollback plan. Communicate cutover dates early. The parallel run is what makes it safe: every difference is found by an automated reconciliation, not by a business user.

▶ **Parallel run + automated reconciliation + rollback**

If they push back:

↳ **"What if old and new outputs don't match?"**
I check whether the old system was wrong too. Often the legacy logic had a bug. Either way, the business owner signs off on which result is correct before cutover.

↳ **"How long do you run in parallel?"**
At least one full business cycle, including month-end, because that's when edge cases show up.

Conflict & stakeholders

Q10. Did you have a conflict with a senior? How did you resolve it?

My senior wanted to keep a full refresh for a large fact table; I believed incremental MERGE was needed because the refresh took 3 hours and was growing. Instead of arguing, I built a PoC on one month of data comparing runtime, cost and correctness, and listed my approach's risks honestly (late-arriving data). We agreed on incremental loads plus a weekly full reconciliation, which covered his correctness concern. I disagreed in private, not in the team meeting, and once we decided I fully committed to the agreed design.

▶ **Bring data, not opinions — solve their real concern**

If they push back:

↳ **"What if the PoC had proved him right?"**
Then we'd have kept the full refresh. The goal was the right design, not winning. That's why I tested it instead of debating it.

↳ **"How do you handle late data in incremental loads?"**
MERGE on the business key with a lookback window of a few days, plus the weekly full reconciliation to catch anything outside it.

Q11. The client / downstream team is upset about your work. How do you handle

it?

Listen fully, don't get defensive — the frustration is about business impact. Confirm the exact issue: which report, which numbers, which deadline. Acknowledge it and give a clear next-update time even without a fix. Fix or work around, explain root cause in plain language, share the prevention. When a sales team got late data three days running, I set up a daily freshness status message and moved their tables earlier in the DAG. Complaints stopped. I don't argue about whose fault it was in that moment. The client needs to see that someone owns it and has a plan.

▶ **Listen → acknowledge → update time → fix → prevent**

> *If they push back:*
>
> ↳ **"What if the problem was actually on their side?"**
> I fix what I can first, then show the evidence calmly, for example their source file arrived late, and propose a fix together, like an arrival SLA or a sensor.
>
> ↳ **"What if you can't fix it quickly?"**
> I give a realistic ETA and a workaround, like yesterday's data with a staleness note, and update them at the time I promised even if there's no news.

Q12. You disagreed with a teammate's code in review. What happened?

A PR used a Python UDF where native Spark functions would work. I commented with the reason — UDFs block Catalyst optimizations and add serialization cost — suggested the **`when` / `regexp_extract`** equivalent, and offered to pair. He changed it; the step ran ~4x faster. Reviews are about the code, never the person, and I don't block on style preferences. Code review is also where I share knowledge, so I explain the reason so the next PR avoids it without my help.

▶ **Explain the why, offer to pair**

> *If they push back:*
>
> ↳ **"What if he had refused to change it?"**
> If it was performance-critical, I'd show a quick benchmark. If it wasn't, I'd approve it and leave it as a suggestion. Not every comment is a blocker.
>
> ↳ **"Are UDFs always bad?"**
> No. When native functions can't express the logic, a pandas UDF with Arrow is a reasonable choice. The rule is native first.

Q13. A stakeholder asks for something urgent that conflicts with your sprint.

Don't say yes or no immediately. Clarify business impact and the real deadline — 'urgent' often means Friday. Make the trade-off visible to my manager and the stakeholder: if this comes in, these items move by these dates. They decide priority; I deliver what's agreed. A genuine production issue goes first, and I communicate what slips. Saying yes to everything silently is how deadlines slip without anyone deciding it, so I make the trade-off explicit.

▶ **Make the trade-off visible, let owners decide**

> *If they push back:*

↳ **"What if your manager says 'do both'?"**
I break both into what's achievable and show the risk. If we still go ahead, I flag it early if something slips rather than at the deadline.

↳ **"Have you ever said no to a stakeholder?"**
I say 'not now' with a reason and a date. A flat no without an alternative damages the relationship.

Q14. Requirements were unclear or kept changing. How did you manage?

The definition of 'active customer' changed twice mid-build. I wrote it down with examples and edge cases and got business sign-off. I built the logic as one configurable rule instead of spread through the code, so the third change was a config change + backfill, not a rewrite. I shared sample outputs early — people correct requirements faster when they see real numbers. Ambiguous requirements are normal in data work. The skill is making them concrete early, before they're baked into code.

▶ **Write it down, make it configurable, show samples early**

If they push back:

↳ **"How do you get business sign-off without delaying work?"**
I build in parallel with the rule isolated, share sample outputs, and get confirmation on real numbers, which is faster than a document review.

↳ **"What if they change it after go-live?"**
Because the rule is configurable and data is versioned, I change it, backfill the affected range, and note the change for consumers.

Ownership, pressure & ambiguity

Q15. Tell me about a time you missed a deadline.

I underestimated a migration because the source had undocumented stored procedures with business logic. Halfway through, I told my manager immediately, gave a revised estimate with the reason, and proposed delivering critical tables first so downstream wasn't blocked. Core tables shipped on the original date, the rest four days later. Now I add a discovery spike before estimating migrations and flag risks the day I find them. The key point is that I told my manager when I discovered the risk, not on the deadline day.

▶ **Flag early, deliver critical path first**

If they push back:

↳ **"Why did you underestimate it?"**
I estimated from the table list without checking the stored procedures. Now I do a short discovery step before committing to dates.

↳ **"How did stakeholders react?"**
Because I flagged it early and protected the critical tables, they could plan around it. No downstream work was blocked.

Q16. You inherited a pipeline with no documentation. What did you do?

Treated it as reverse engineering. Traced lineage from final tables back to sources via the code and the Airflow DAG, documenting each step's inputs, outputs and rules. Added logging and row counts per stage. Captured a baseline of current outputs before touching any logic. Wrote a runbook for failures, reruns and backfills — the next joiner was productive in days. The baseline is the most important step: without it, you can't prove that a change didn't break something.

▶ Lineage → baseline → then change → runbook

> If they push back:
>
> ↳ **"How did you find hidden business rules?"**
>
> I compared input and output at each step with row counts and sample keys. Where data changed unexpectedly, I read the code and confirmed the rule with the business.
>
> ↳ **"Why not just rewrite it?"**
>
> A rewrite without understanding the existing rules is how you lose logic people depend on. Understand first, then refactor in small, tested steps.

Q17. You found a problem that wasn't your responsibility. What did you do?

Debugging my job, I noticed an upstream table silently dropped late events because of a strict ingestion-time filter. Not my pipeline, but many teams depended on it. I documented evidence with sample records and counts, raised it with the owners and suggested an event-time window with a watermark. Fixed the same week. I didn't just report a bug. I brought proof and a suggested fix, which made it easy for them to act.

▶ Evidence + proposed fix = fast response

> If they push back:
>
> ↳ **"Why didn't you just fix it yourself?"**
>
> It was their pipeline and other teams depended on it. Changing it without the owners could break something I didn't know about.
>
> ↳ **"What if they hadn't prioritized it?"**
>
> I'd escalate through my lead with the impact numbers, and add a temporary workaround on our side meanwhile.

Q18. How do you work under pressure — release + production issue together?

Separate them and make priorities explicit: a user-facing production issue first. Give stakeholders a short status with ETA so pressure becomes a visible plan. Ask for help early instead of quietly risking both. Afterwards, find why they collided — usually a missing test or a release too close to a peak — and fix that. I make the plan visible because a stakeholder who can see the plan doesn't need to keep asking for updates.

▶ Prioritise out loud, ask for help early

> If they push back:
>
> ↳ **"How do you decide what comes first?"**
>
> User or revenue impact first, then contractual SLAs, then internal deadlines.

↳ **"What if both are equally critical?"**
I escalate early to my manager so the decision is made by the right person, and ask for another engineer to take one.

Leadership, design & growth

Q19. Tell me about an improvement you drove without being asked.

We handled the same failures manually every week: reruns, late files, small-file buildup. I grouped recurring alerts and automated the top three: retries with backoff for transient errors, a file-arrival sensor instead of fixed schedules, and a weekly **OPTIMIZE / VACUUM** *job. Manual on-call interventions dropped by about half; I shared it as a reusable module. I chose what to automate from data: the alert history showed which three failure types made up most of the manual work.*

▶ **Automate the top recurring alerts**

If they push back:

↳ **"How did you measure the 50% drop?"**
Count of manual interventions from PagerDuty incidents and on-call notes, four weeks before vs. four weeks after.

↳ **"Doesn't auto-retry hide real problems?"**
Retries are only for transient error types with a small limit. Anything else fails fast and alerts.

Q20. How do you mentor juniors or onboard new team members?

A real but low-risk task in week one, like adding a DQ check, so they learn deployment end to end. Pair on their first production issue instead of fixing it for them. In reviews, explain the why — partitioning, idempotency, skew. Success = how soon they can handle on-call alone. The goal is independence: the junior should be able to handle the work without me in a few weeks.

▶ **Real task in week 1, pair on first incident**

If they push back:

↳ **"What if a junior keeps making the same mistake?"**
I look for a gap in the process, like a missing check or unclear docs, and fix that, then pair on one more case.

↳ **"How do you balance mentoring with your own work?"**
Short, regular pairing slots instead of constant interruptions, plus good runbooks so common questions answer themselves.

Q21. Why should we hire you for a senior data engineer role?

I've built and supported production pipelines across e-commerce, automotive, finance and industrial IoT — batch and streaming, on-prem and cloud. I own pipelines end to end: design, tuning, data quality, on-call and cost. I fix the process behind a problem, not just

Sagar

the problem, and I communicate clearly with business stakeholders. Back every claim with one real example from your career, because a follow-up will ask for one.

▶ **End-to-end ownership + process fixes + communication**

> *If they push back:*
>
> ↳ **"What is your biggest weakness?"**
> *I used to dig into problems alone for too long. Now I set a time limit and ask for help or escalate early.*
>
> ↳ **"Why are you looking to move?"**
> *I want larger-scale ownership: designing platforms and leading engineers, not only building pipelines. Keep it positive about your current role.*

Before I walk in

- ✓ Real numbers I can defend
- ✓ 90 sec – 2 min per answer
- ✓ "I" for my work, "we" for team wins
- ✓ Name the exact tool / config
- ✓ Always end with prevention

- × Blaming senior / teammate / client
- × "I never had a conflict / mistake"
- × "I optimized it" — no cause, no number
- × Claims I can't explain deeply

— **Sagar**

Sagar